

sVB Debager

Debugovanje sVB projekata

Kada pritisnete F5 da biste pokrenuli projekat, sVB kompajlira kôd i kreira Exe datoteku projekta, koja se čuva u folderu bin u folderu projekta, sa svim potrebnim datotekama kopiranim u ovaj folder, kao što su xaml, slike i txt datoteke, plus datoteka SmallVisualVasicLibrary.dll koja sadrži sVB biblioteku koda. Nakon kreiranja exe datoteke, sVB je pokreće, tako da se vaš program pokreće i izvršava vaš kôd.

Dakle, sVB IDE zapravo ne komunicira sa pokrenutim programom, osim što ga može zatvoriti kada kliknete na dugme End Program – *Završi program* koje se pojavljuje na ekranu.

Dakle, šta se dešava kada se dogodi greška u vašem programu? Pa, sVB vam može reći o greškama koje može otkriti u vašem kodu, a sVB biblioteka vas može upozoriti na greške koje se dešavaju kada se program pokreće, ali sVB IDE vam ne može reći gde se tačno greška dogodila u vašem kodu.

Pronalaženje i ispravljanje grešaka u programu naziva se debugovanje.

Programski jezici pružaju alat koji vam pomaže da debugujete svoj program, a to je: debager. Srećom, Small Visual Basic je konačno dobio debager počev od verzije 3.0, koji je sličan VS .NET debageru koji se koristi sa VB .NET i C#.

U narednim odeljcima ćemo saznati o ovom debageru i kako ga koristiti, ali prvo hajde da malo dublje pogledamo greške i zašto se javljaju, kako bismo ih mogli ispraviti.

Vrste grešaka

Postoje tri vrste grešaka

1. Greške kompajlera

Ove greške se mogu desiti dok sVB kompajlira vaš kôd, zbog sintaksičkih grešaka, kao što je nedostatak operatora ili pogrešno kucanje ključne reči. Programski jezici očekuju da napišete kôd u ispravnom obliku, a ako to ne učinite, daju vam sintaksičke greške.

Semantičke greške se takođe mogu desiti, kao kada pokušate da pročitate promenljivu kojoj još niste postavili vrednost, ili pozovete potprogram koji ne postoji, ili prosledite pogrešan broj argumenata funkciji... itd.

sVB kompajler detektuje sintaksičke i semantičke greške pre kompajliranja exe datoteke, i ako pronađe bilo koju od njih, proces kompajliranja se otkazuje, a lista grešaka se prikazuje na dnu uređivača koda.



Možete dvaput kliknuti na bilo koju grešku na listi da biste istakli deo koda koji ju je izazvao. Morate ispraviti sve ove greške, a zatim pokušati ponovo da pokrenete program.

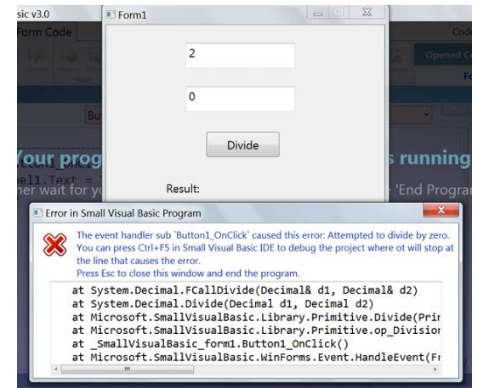
2. Greške tokom izvršavanja

Ove greške se mogu desiti dok se program izvršava, zbog izvršavanja pogrešne operacije. Kompajler ne može da detektuje takvu vrstu grešaka pre pokretanja programa, jer zavise od vrednosti promenljivih i korisničkih akcija.

Na primer, ako zamolite korisnika da unese dva broja u dva tekstualna polja da bi ih podelio i

prikazao rezultat, korisnik može da unese 0 u drugo tekstualno polje, a ako ne napišete kôd za obradu takvog slučaja, to će izazvati grešku deljenja sa nulom tokom izvršavanja:

Kada se dogodi greška tokom izvršavanja, program će prikazati poruku o grešci koja opisuje grešku i obaveštava vas da će program biti prekinut nakon što zatvorite poruku. Takođe vam savetuje da pokrenete program u režimu za otklanjanje grešaka, kako biste mogli da otkrijete liniju koda koja je izazvala ovu grešku, što je važan korak koji vam omogućava da identifikujete izvor greške kako biste je mogli ispraviti.



Možete otkloniti greške u programu klikom na dugme Debug – *Otklanjanje grešaka* ili pritiskom na Ctrl+F5.

3. Logičke greške

Ove greške ne može da otkrije ni kompajler niti se mogu otkriti za vreme izvršavanja programa, jer ne ometaju izvršavanje programa, ali mogu prouzrokovati pogrešne rezultate ili učiniti da se program ne ponaša pravilno. Ove greške mogu biti uzrokovane greškom u algoritmu programa, tako da ste vi i korisnici programa odgovorni za otkrivanje takvih grešaka.

Da biste ispravili takve greške, morate pregledati svoj kôd i pronaći šta nije u redu u njegovoj logici ili dodati nedostajući kôd koji obrađuje neke slučajeve. Ponekad nije lako otkriti uzrok logičke greške, pa ćete možda morati da pokrenete program u režimu za otklanjanje grešaka i pratite kôd i vrednosti promenljivih da biste videli šta se dešava.

Dakle, možete zaključiti da je program za otklanjanje grešaka važan da bi vam pomogao da ispravite greške tokom izvršavanja i logičke greške.

Komande programa za otklanjanje grešaka



Program možete pokrenuti u režimu za otklanjanje grešaka klikom na dugme Debug, koje možete videti na traci sa alatima dizajnera obrazaca pored dugmeta Run (*slika desno iznad*).

Ovo dugme se takođe prikazuje na traci sa alatima uređivača koda (*slika desno*).



Ili možete jednostavno pritisnuti Ctrl+F5 da biste pokrenuli debager.

Traka sa alatima uređivača koda takođe prikazuje dugme Step Into – Uđi u  koje pokreće debager i pauzira ga na prvoj liniji programa.

Imajte na umu da će se traka sa alatima uređivača koda promeniti kada pokrenete program u režimu debugovanja, gde će se prikazati neka nova dugmad koja vam daju potpunu kontrolu nad debagerom. Sledeća slika prikazuje programski deo trake sa alatima u režimu debugovanja (*slika desno*):



Da vidimo šta ta dugmad mogu da urade:

- **Run – Pokreni (F5):**  Nastavlja pokretanje debagera ako je trenutno pauziran.

- **Pause – Pauziraj (Ctrl+F11):**  Primorava program da pauzira na liniji koda koja će se uskoro izvršiti. Ova linija će biti označena zlatnom pozadinom, kao što možete videti na slici ispod.

Imajte na umu da kada je program u režimu prekida, možete umetnuti kursor na bilo koju

promenljivu ili izabrati bilo koji važeći izraz da biste prikazali njegovu trenutnu vrednost. Na slici, iskačuće objašnjenje vam govori da je vrednost promenljive SpeedY u ovom trenutku 10.

- End program – Završi program (Shift+F5):  Završava pokrenuti program. Traka sa alatima za uređivanje koda će se vratiti u normalno stanje, prikazujući skrivenu dugmad i sakrivajući dodatna dugmad za otklanjanje grešaka.

- Step into – Uđi u (F11):  Ako je debager pauziran (u režimu prekida), ova komanda će izvršiti trenutni red i pauzirati u sledećem redu. Ako trenutni red poziva bilo koji drugi deo programa, kao što je potprogram ili funkcija, čak i u drugoj datoteci, debager će pauzirati u prvom redu tog dela.

Na sledećoj slici, program je pauziran u redu koji poziva potprogram Test. Ako kliknete na Step Into, pauziraće se u redu na koji pokazuje crvena strelica, što je prvi red potprograma Test.

Možete više puta kliknuti na Step Into da biste izvršili kôd red po red, što vam omogućava da vidite kako se kôd izvršava i proverite vrednosti promenljivih. Ovo vam ne samo da može pomoći da otkrijete izvor greške koja dovodi do greške u vašem programu, već vam može omogućiti i da analizirate i razumete kôd nekih programa koje niste vi napisali. Na primer, pokušajte da uđete u igru Cars u folderu sVB primeri i vidite kako funkcioniše. Ovo je prvi korak koji vam može pomoći da rešite probleme koje sam ostavio u igri (kao što su drugi automobili koji prelaze preko vašeg automobila ili vaš automobil izlazi sa trkačke staze).

Isto možete učiniti i sa drugim primerima, kao što je igra Tetris.

Imajte na umu da će vas izvršavanje linije EndSub ili EndFunction vratiti na liniju koja je pozvala trenutnu potprogramu funkcije.

- Step Over – Pređi preko (F10):  Ako je debager pauziran (u režimu prekida), ova komanda će izvršiti trenutnu liniju i pauzirati u sledećem redu. Ako trenutna linija pozove bilo koje potprograme ili funkcije, debager će ih izvršiti, ali se neće zaustaviti ni na jednoj od njih, čak i ako sadrže tačke prekida.

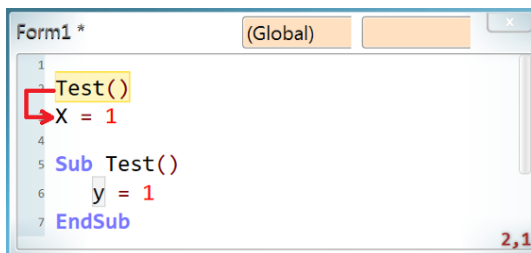
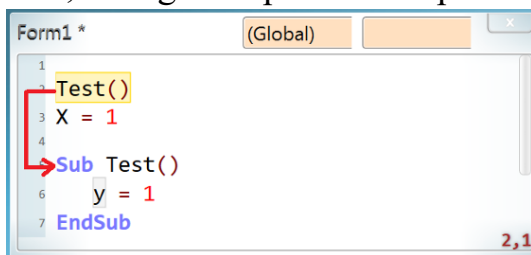
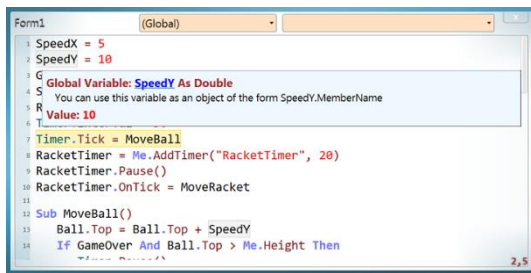
Na sledećoj slici, program je pauziran u liniji koja poziva potprogramu Test. Ako kliknete na Step Over, pauziraće se u liniji na koju pokazuje crvena strelica, što je tačno sledeći red. Debager će izvršiti potprogram Test, ali neće pauzirati unutar njega. Zato kažemo da je debager prešao preko poziva potprograma.

- Block Over – Preskoči blok (Ctrl+F10):  Ako je debager pauziran (u režimu prekida), ova komanda će izvršiti trenutni red, prelazeći preko njegovih unutrašnjih blokova, ako ih ima.

Na primer: Ako je trenutni red zaglavlje If naredbe, klikom na ovo dugme izvršiće se blok if naredbe, ali debager neće pauzirati ni na jednu naredbu u telu If naredbe ili bilo kom telu bilo kog povezanog tela ElseIf i Else, i pauziraće samo na redu pored reda EndIf.

Isto se dešava ako je trenutni red ElseIf ili Else.

Sledeća slika prikazuje program Ball Game u režimu prekida. Pauziran je na If naredbi, a ako kliknete na Block Over, pauziraće se na redu na koji pokazuje crvena strelica.



Debager će uraditi isto ako je trenutna linija početak bilo kog drugog bloka koda, kao što su naredbe For, ForEach i While, gde će debager pauzirati na liniji pored kraja bloka, kao što su Next i Wend.

Ako trenutna linija poziva potprogram ili funkciju, ova komanda će je preskočiti.

- Step Out – Izađi (F12):  Ako je debager pauziran (u režimu prekida), ova komanda će izvršiti trenutni potprogram i pauzirati na liniji koja ju je pozvala.

Na sledećoj slici, program je pauziran na liniji unutar potprograma Test. Ako kliknete na Step Out, debager će izvršiti petlju for i izaći iz potprograma Test, a zatim će pauzirati na liniji na koju pokazuje crvena strelica, što je linija koja je pozvala potprogram Test.

- Block Out – Izađi iz bloka (Ctrl+F12):  Ako je debager pauziran (u režimu prekida), ova komanda će izvršiti telo bloka koda koji sadrži trenutnu liniju i pauzirati na liniji pored kraja bloka.

Na sledećoj slici, program je pauziran u redu unutar for petlje. Ako kliknete na Block Out, debager će nastaviti izvršavanje for petlje, a zatim će se pauzirati u redu na koji pokazuje crvena strelica, što je prvi red koji sledi posle Next.

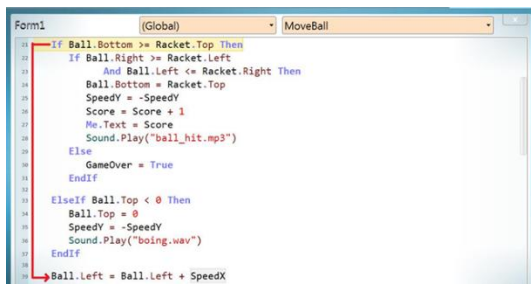
Imajte na umu da ako je debager pauziran u liniji For ili Next, klik na Block Out će izaći iz roditeljskog bloka koji sadrži For petlju, što je Test potprograma u našem primeru, tako da ćete se vratiti na liniju pozivaoca.

- Breakpoint – Tačka prekida (F9):  Ovu komandu možete koristiti u bilo kom trenutku (čak i tokom debugovanja) da biste dodali tačku prekida u trenutnoj liniji ili uklonili postojeću.

Imajte na umu da takođe možete promeniti tačku prekida bilo koje linije dvostrukim klikom na njenu marginu.

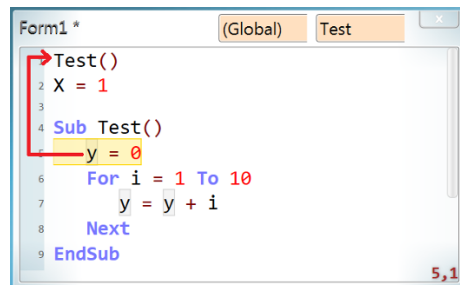
Tačke prekida mogu vam uštedeti vreme, jer vam omogućavaju da pauzirate debager u linijama koje izaberete, bez potrebe da ulazite u kôd od prve linije.

Sve što vam je potrebno je da dodate tačku prekida u liniju, a zatim pokrenete program u režimu debugovanja. Debager će izvršiti kôd programa i ako naiđe na liniju sa tačkom prekida, pauziraće se kako biste mogli da pregledate vrednosti promenljivih i prođete kroz kôd.



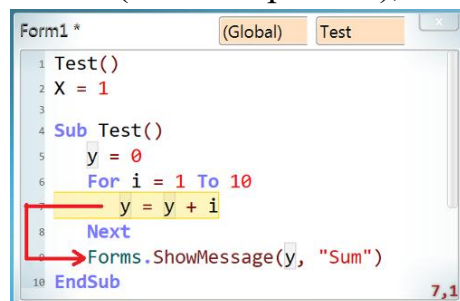
```

1 If Ball.Bottom >= Racket.Top Then
2   If Ball.Right >= Racket.Left
3     And Ball.Left <= Racket.Right Then
4       Ball.Bottom = Racket.Top
5       SpeedY = -SpeedY
6       Score = Score + 1
7       Me.Text = Score
8       Sound.Play("ball_hit.mp3")
9   Else
10    GameOver = True
11  EndIf
12 ElseIf Ball.Top < 0 Then
13   Ball.Top = 0
14   SpeedY = -SpeedY
15   Sound.Play("boing.wav")
16 EndIf
17 Ball.Left = Ball.Left + SpeedX
  
```



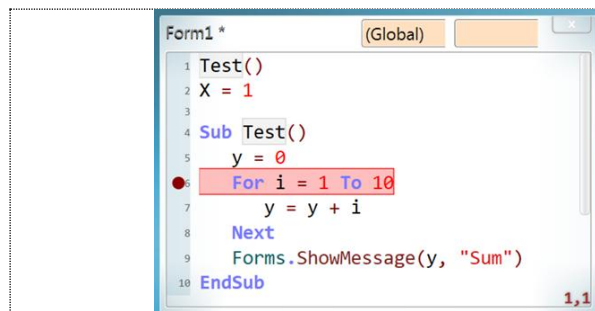
```

1 Test()
2 X = 1
3
4 Sub Test()
5   y = 0
6   For i = 1 To 10
7     y = y + i
8   Next
9 EndSub
  
```



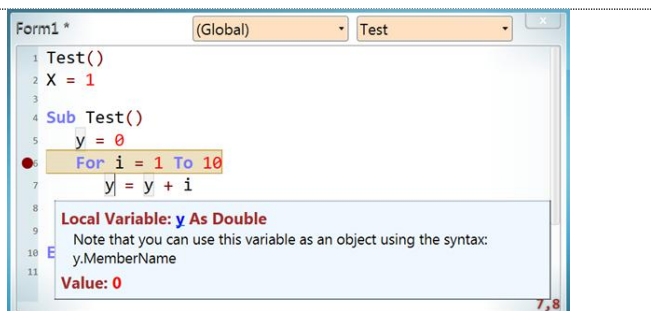
```

1 Test()
2 X = 1
3
4 Sub Test()
5   y = 0
6   For i = 1 To 10
7     y = y + i
8   Next
9   Forms.ShowMessage(y, "Sum")
10 EndSub
  
```



```

1 Test()
2 X = 1
3
4 Sub Test()
5   y = 0
6   For i = 1 To 10
7     y = y + i
8   Next
9   Forms.ShowMessage(y, "Sum")
10 EndSub
  
```



```

1 Test()
2 X = 1
3
4 Sub Test()
5   y = 0
6   For i = 1 To 10
7     y = y + i
8   Next
9   Forms.ShowMessage(y, "Sum")
10 EndSub
  
```

Local Variable: y As Double
Note that you can use this variable as an object using the syntax: y.MemberName
Value: 0

Možete ukloniti tačku prekida u režimu prekida ili dodati nove i kliknuti na Run da biste nastavili sa debugovanjem i dozvolili debageru da se zaustavi na sledećoj tački prekida.

Prekid kod grešaka

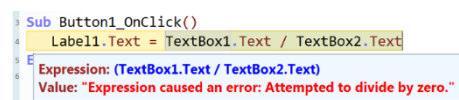
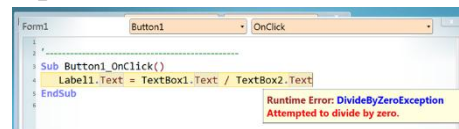
Kada pokrenete program u režimu za otklanjanje grešaka i nađete na grešku tokom izvršavanja, on će se zaustaviti na liniji koja uzrokuje grešku, što će vam omogućiti da je ispravite.

Imajte na umu da ne možete menjati kôd dok se program izvršava, jer će sve datoteke koda projekta biti samo za čitanje. Dakle, kada se debager zaustavi kôd greške, možete samo pročitati poruku o grešci i proveriti vrednosti promenljivih, ali ne možete ništa promeniti, niti nastaviti izvršavanje programa, jer će se odmah prekinuti.

U primeru deljenja sa nulom koji smo ranije pomenuli, ako pokrenemo program u režimu za otklanjanje grešaka, sledeća slika vam pokazuje kako se debager ponaša kada se dogodi greška:

Možete izabrati izraz `TextBox1.Text / TextBox2.Text` da biste videli njegovu vrednost, što će vam dokazati da je to izvor greške:

Takođe možete umetnuti kursor u svojstvo `Text` elementa `TextBox1` ili `TextBox2` da biste prikazali njihove vrednosti. Ali jedina akcija koju sada možete preduzeti je da kliknete na dugme `End Program`, kako biste mogli da ispravite ovu grešku. Jedno moguće rešenje je pisanje `If` naredbe za proveru vrednosti `TextBox2.Text` i prikazivanje poruke korisniku da bi se obavestilo da 0 nije prihvaćena vrednost. Možete videti kako se to radi u primerima kalkulatora u sVB folderu sa primerima.



Otklanjanje grešaka u test funkcijama

Debager može da otklanja greške u test funkcijama koje pozivate pomoću metoda `Form.RunTests`, `Form.RunGlobalTests`, `UnitTest.RunAllTests`, `UnitTest.RunFormTests` ili `UnitTest.RunGlobalTests`.

Ako uđete u bilo koju od tih metoda, debager će preći u prvu metodu testiranja, a nakon što se ona završi, preći će u drugu, treću i tako dalje.

Takođe možete dodati tačke prekida u bilo koju funkciju testiranja, da biste primorali debager da se zaustavi na njima kada pokrećete testove.

Kako debager radi?

Važno je razumeti kako debager radi, kako bi se izbegli neki problemi koji se mogu desiti prilikom njegovog korišćenja.

Kada pokrenete debager, sVB kompajlira kôd i izgrađuje .exe datoteku, ali je ne pokreće. Proces kompajliranja je neophodan da bi se osiguralo da kôd nema sintaksičke ili semantičke greške i da bi se izgradilo sintaksno stablo koje debager koristi za izvršavanje programa.

S druge strane, izgradnja .exe datoteke nije neophodna za debager, ali osigurava da fascikla bin uvek sadrži poslednju ažuriranu verziju vaše izvršne aplikacije, čak i nakon što debugujete projekat. Ova .exe datoteka je čista verzija projekta koja nema simbole za debugovanje i nije ni na koji način povezana sa debagerom, tako da je možete koristiti i distribuirati kako želite.

Ali možete se pitati: Zašto sVB, po default, ne radi u režimu debugovanja?

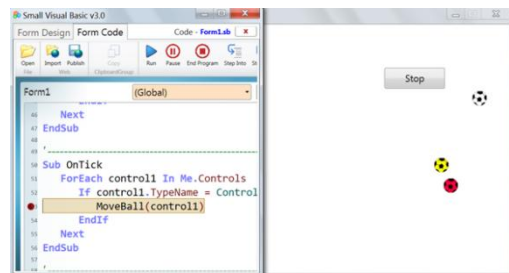
Zapravo, debager može da radi malo sporije od .exe datoteke i to se može primetiti kada debugujete aplikaciju koja izvršava mnogo pokreta i okretanja kornjače čak i kada isključite animaciju

kornjače. To je zato što:

1. Debager interpretira svaku instrukciju tokom izvršavanja, pa izvršava potprograme, if naredbe i petlje, i procenjuje promenljive i pozive metoda. Ako se ista naredba pozove više puta (linijski for petlje ili pozivi potprograma), biće ponovo interpretirana svaki put, što može biti sporije od izvršavanja kompajliranog koda niskog nivoa.

2. Debager koristi rečnik visokog nivoa za čuvanje vrednosti promenljivih i parametara potprograma, što je sporije od korišćenja steka niskog nivoa tokom izvršavanja .exe datoteke.

3. Debager radi u istom procesu kao i sVB IDE, koji zahteva pozivanje svakog obrađivača događaja u novoj niti kako bi IDE ostao odgovoran u režimu pauze i omogućio vam da vidite promene na formi dok prolazite kroz njen kôd. Pokušajte da postavite tačku prekida u potprogramu MoveBall u jednom od projekata Ball u sVB folderu sa primerima, prikažite i formu i sVB IDE na ekranu jedan pored drugog i pogledajte kako se loptice kreću korak po korak na formi:



Kreiranje nove niti za svaki obrađivač događaja može biti skupo, posebno kada koristite tajmere, jer će svako otkucavanje svakog tajmera kreirati novu nit.

Imajte na umu da debager sprečava kreiranje nove niti za isti obrađivač događaja dok se prethodna ne završi. Takođe suspenduje sve niti u režimu prekida, tako da možete da snimate stanje programa u trenutku prekida i da se krećete kroz linije koda bez prekida zbog otkucavanja događaja.

Ovo vam omogućava da debugujete igre (gde se tajmeri često koriste), a da se igra ne završi dok ste u režimu prekida!

Ali zapamtite da je forma i dalje odzivna u režimu prekida, pa šta ako možete da kliknete na bilo koje dugme dok je debager pauziran?

U tom slučaju, debager će vam prikazati poruku koja vas podseća da ste u režimu debugovanja i prebaciće vas na liniju koja će se uskoro izvršiti u uređivaču koda.

Korišćenje niti za događaje takođe može izazvati neke probleme prilikom debugovanja događaja koje pokreće kôd, kao kada promenite svojstvo CheckBox.Checked iz koda, što pokreće Događaj OnCheck. Zato budite oprezni kada koristite debager u takvim situacijama.

Možete zaključiti da ne bi trebalo da koristite debager osim ako vam nije potrebno da razumete kako kôd funkcioniše (ili ga objasnite studentima), ili da otkrijete izvor greške, u suprotnom je brže pokrenuti kompajlirani program.

Najbolja praksa za otklanjanje grešaka

1. Trebalo bi mudro da koristite debager, jer prebrzo i previše koraćanje kroz kôd (kao kada držite prst na tasteru F11 unutar dugačke for petlje) može dovesti do toga da neke niti uđu u mrtvu blokadu koja sprečava izvršavanje nekih delova programa, i ako se to desi, trebalo bi da završite debager i počnete ispočetka.

2. Kada obrađujete događaje koji se mogu često pokretati u kratkom vremenskom periodu (kao kada korisnik pritisne taster – strelicu nagore da bi ubrzao automobil u igri), pokušajte da izbegnete nepotrebna izvršavanja ovih obrađivača ako je moguće, posebno ako je kôd obrađivača dugačak ili sadrži petlje ili poziva druge potprograme, jer to može blokirati nit debagera. Na primer, obrađivač Form1_OnKeyDown u igri Car poziva neke potprograme da bi podesio neke promenljive koje se koriste u obrađivaču OnTick tajmera da bi se ubrzao automobil nagore ili nadole, ili pomerio

levo ili desno. Tajmer nastavlja to da radi dok korisnik ne spusti prst, tako da obrađivač Form1_OnKeyUp resetuje te promenljive da bi zaustavio ubrzavanje ili spuštanje i kretanje levo ili desno. Dakle, korisnik će držati prst na jednom od tastera sa strelicama (recimo taster Nagore da bi ubrzao automobil), što znači da će Form1_OnKeyDown biti pozivan 50 ili 100 puta u sekundi, što je u redu u režimu izvršavanja, ali će preopteretiti debager jer će svaki poziv kreirati novu nit, a svaka nit će pokušati da blokira nove dok se ne završi, a to će prekinuti igru!

Dakle, trebalo bi da izbegavate takvu situaciju. Zapravo, rešenje je jednostavno: Samo izađite iz obrađivača ako se izvršava komanda za kretanje ili kočenje, jer obrađivač OnKeyDown zapravo neće ništa raditi dok tajmer obavlja posao. Ovo je kôd koji se koristi za rešavanje problema sa debugovanjem, a takođe poboljšava performanse igre u režimu izvršavanja:

```
Sub Form1_OnKeyDown()  
If StartMove Or Braking Then  
Return  
EndIf  
    ' Ostatak koda ovde  
EndSub
```

3. Izvršavanje događaja Form.OnClosing u posebnoj niti može izazvati problem u režimu za otklanjanje grešaka kada podesite Event.Cancel na True da biste otkazali zatvaranje forme, jer glavna nit nastavlja da izvršava svoj kôd i možda neće pročitati vrednost koju ste postavili, tako da se forma može ipak zatvoriti! Da bi rešio ovo, debager će primorati glavnu nit da sačeka 100 milisekundi, što znači da morate da napišete:

```
Event.Handled = True
```

na početku koda za rukovanje i da izbegnete izvršavanje petlji ili pozivanje dugotrajnih funkcija pre toga kako biste bili sigurni da će se izvršiti pre nego što istekne kašnjenje od 100 ms.

4. Možete kreirati nove niti da biste paralelno pokretali neke zadatke vašeg programa. Ali u režimu za otklanjanje grešaka, niti se neće kreirati, kako biste mogli da pratite kôd. U ovom slučaju, vaš program može biti sporiji i neka njegova logika možda neće pravilno funkcionisati.